ROBOT NAVIGATION

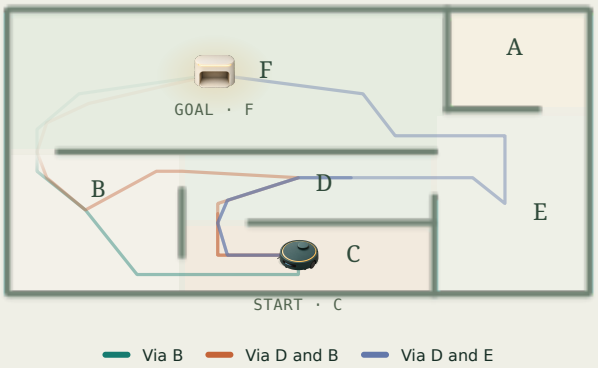
LEARNING TO NAVIGATE

How do robots *find the goal?*

A robot needs to reach a target room. How can it learn which doorway to choose?

I use Q-learning to learn from the moves it tries. Then I change the start and target to see what the robot learns differently.

Chenglong Zhang
50068179 · IOTA 5201 · Fall 2026



Three possible choices through the experimental map. The robot stays in C and the dock in F; faint moving shadows follow the routes and fade at the destination. These are illustrative choices, not recorded training runs.

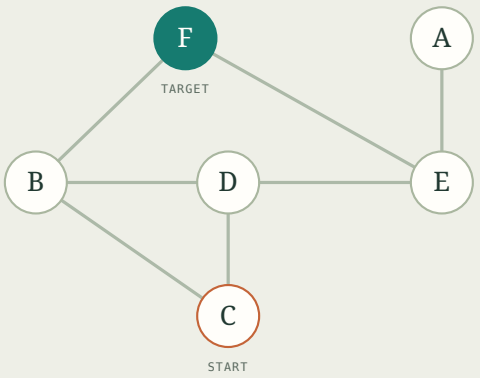
THE PROBLEM

Reach the target. Learn which move to take.

The robot can move between rooms through doorways. At each step, it knows its current room and the moves available there. It needs to learn which move will help it reach the target. I model each room as a **state** and each move into a neighboring room as an **action**.

For my first experiment, the robot starts in **C** and the target is **F**. I read these positions from the robot and dock in the assignment's Figure 1. The second experiment follows the assignment's stated change: start in **A** and make **C** the target.

For example, from C the robot can enter B or D. In D, it can move to B, C, or E. Which of those choices brings it closer to the target? The robot will learn action values from experience rather than being given the answer.



The map used in my experiments. Each circle is a room; a line means movement is allowed in both directions. C and F mark the first start and target. The allowed room pairs are A–E, B–C, B–D, B–F, C–D, D–E, and E–F.

I will explain the learning rule, examine C → F, repeat it for A → C, and compare the results. Additional experiments come afterwards.

The assignment provides a schematic room drawing rather than a transition matrix. The B–C doorway is included, as clarified from Figure 1. The room graph and both diagrams use the same seven connections. Once a room is the target, the episode ends on entry; the robot does not take further actions there.

JUMP TO

01 Motivation & method 02 Environment & setup 03 Results 04 Comparison & analysis 05 Takeaways & references

02 Training settings Implementation choices

03 C → F tables C → F plot A → C tables A → C plot Reference & convergence Saved Q-tables

04 Required comparison Route length & learning Exploration & accuracy Pilot settings

PART 01

Motivation and method

I give the robot a reward of **1 when it reaches the target** and **0 for every other move**. Q-learning stores an estimate for every allowed room-and-move pair. This estimate, called a **Q-value**, describes how much future reward the robot expects from that move if it then acts optimally.

A larger Q-value makes a move more attractive. Future rewards are discounted, so reaching the target sooner is worth more than taking a long detour. These values are **discounted reward estimates, not probabilities**.

After a move, the robot compares its old estimate with a new target: the reward it just received, plus the discounted best estimate in the next room. It moves the old estimate partway towards that target. On reaching the goal, there is no next action, so the target is simply the reward of 1.

Q-LEARNING UPDATE

$$\text{new estimate} = \text{old estimate} + \alpha \times (\text{target} - \text{old estimate})$$

For an ordinary move:

target = reward + γ × best next Q

For a move into the goal:

target = reward

α, the learning rate, controls how much an estimate changes.
γ, the discount factor, reduces the value of delayed rewards.

To choose a move, the robot usually follows its highest current Q-value. Sometimes it tries a random legal move instead. This is **ε-greedy exploration**: ε is the probability of making that random choice. Without trying alternatives, the robot could miss useful routes.

The update always uses the best legal next Q-value, even when exploration makes the robot choose a different action. That is why Q-learning is called an **off-policy** method [1]. In symbols, the ordinary update is $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{b \in A(s')} Q(s',b) - Q(s,a)]$. Terminal moves omit the future-value term.

ONE REWARD, LEARNED BACKWARDS

How does reaching F teach the earlier move from C?

Start with every Q-value at zero and imagine the robot takes C → B → F. The first C → B move changes nothing yet: it earns zero and B has no learned value. Reaching F then gives a reward of 1.

Reaching the goal: B → F

The target is the terminal reward, 1. With a learning rate of 0.1, Q(B,F) moves one tenth of the way from 0 towards 1.

$$0 + 0.1 \times (1 - 0) = \mathbf{0.100}$$

On the next attempt: C → B

Now B's best learned value is 0.1. C → B still earns zero immediately, but its target includes that future reward estimate, discounted by 0.9.

$$0 + 0.1 \times (0 + 0.9 \times 0.1 - 0) = \mathbf{0.009}$$

The point: a move can become valuable even when it earns no reward itself. Repeated visits let the goal reward influence earlier decisions. The small 0.009 estimate is a first update; it is not the final value of the move.

This is a hand-calculated teaching sequence using the experiment's fixed $\alpha = 0.1$ and $\gamma = 0.9$. It is not a claim about the order of actions in a particular saved run.

PART 02

Environment and experimental setup

I train the two required configurations separately. Every run starts with a fresh Q-table: all allowed moves initially have value zero. Nothing learned in C → F is carried over into A → C.

One **episode** is one attempt to reach the goal. It starts in the configured room and ends when the target is reached. A 100-move cap stops an attempt if it runs too long. That cap is a sampling limit, not a new goal, so its final nonterminal update still includes a future value.

Random exploration can change the learning path. I therefore repeat each experiment with **20 random seeds**. The tables below show the preselected run with seed 0, while the plots also show variation across all 20 runs.

The shared training settings. I keep these fixed so that the two experiments use the same learning procedure.

SETTING	USED IN BOTH REQUIRED CASES
Initial values	Zero for every allowed move
Learning rate α	0.1, kept constant
Discount γ	0.9
Exploration ε	0.5, kept constant
Training budget	20,000 episodes per run
Repeated runs	20 seeds, numbered 0–19
Episode cap	100 moves

▾ **Implementation choices**

Only adjacent-room actions are available; blocked moves are masked rather than assigned numeric Q-values. The goal has no outgoing actions and no self-loop. Greedy ties during training are broken uniformly at random. Evaluation selects the alphabetically first action among exact ties and uses none of the training random-number stream.

All seeds are 0–19, using NumPy `default_rng` with PCG64. The recorded training environment is Python 3.13.12 and NumPy 2.5.3. The same seed list is reused across configurations, but different trajectories consume random draws differently. I train for the full budget rather than stopping when a plot appears flat.

PART 03

Results for both configurations

I need to answer two questions: **which moves does the robot learn to choose, and do its value estimates settle down?** The Q-tables answer the first question. The learning plots help answer the second.

How to read a Q-table. A row is the room the robot is in. A column is the room it could move into. Read across one row and choose a largest value. A dash means that move is unavailable; it is not a zero-valued choice. The target row is all dashes because the robot stops there.

How to read the plots. The first panel follows the robot's best estimate for its first move. The second checks the largest error anywhere in the table. I can calculate exact values for this small deterministic map, so a falling error line means that the learned table is getting closer to the correct one. These exact values are used only for checking, never to teach the learner its answer.

> **What counts as convergence here?** I require the largest absolute Q-value error over every allowed move to stay below **0.001** for at least 100 consecutive episodes through the end of the run. This is a finite-run accuracy check, not a proof of the general Q-learning convergence theorem. All plots use the median across 20 seeds; shaded bands span the 10th–90th percentiles and show variation between runs, not confidence intervals.

EXPERIMENT 1

Can the robot learn to reach F from C?

Compare the two tables below. Initially every allowed move is worth zero, so the robot has no learned preference. Afterwards, moves that can lead to F sooner have larger values. The F row stays empty because reaching F ends the episode. These are the saved initial and final tables from seed 0.

Before learning

Initial Q-table · episode 0 · C → F · seed 0

S / A	A	B	C	D	E	F
A	–	–	–	–	0	–
B	–	–	0	0	–	0
C	–	0	–	0	–	–
D	–	0	0	–	0	–
E	0	–	–	0	–	0
F	–	–	–	–	–	–

After learning

Final Q-table · episode 20,000 · C → F · seed 0

S / A	A	B	C	D	E	F
A	–	–	–	–	0.900	–
B	–	–	0.810	0.810	–	1.000
C	–	0.900	–	0.810	–	–
D	–	0.900	0.810	–	0.900	–
E	0.810	–	–	0.810	–	1.000
F	–	–	–	–	–	–

> Read row **C** in the learned table: moving to B gives approximately **0.900**, while moving to D gives **0.810**. The B–C doorway makes C → B → F the shorter choice, so B is preferred.

Values are rounded to three decimals; zeros are shown as 0. Full-precision files: initial table · final table · visit counts.

Following the highest learned values gives this route:

```
C → B → F
```

It takes **2 moves**, matching a shortest route in the map. This is a greedy evaluation: the robot follows its learned values without deliberately exploring. Exact ties during evaluation use the alphabetical rule; tiny numerical differences can choose between equally short routes.

Did the values settle down?

The first panel shows the best estimate for the first move rising from zero towards **0.90**. That is the correct discounted value of reaching the goal in 2 moves. The second panel checks *every allowed move*, including moves away from this route: as its error falls, the whole table becomes more accurate. Its dashed line marks my 0.001 accuracy threshold.

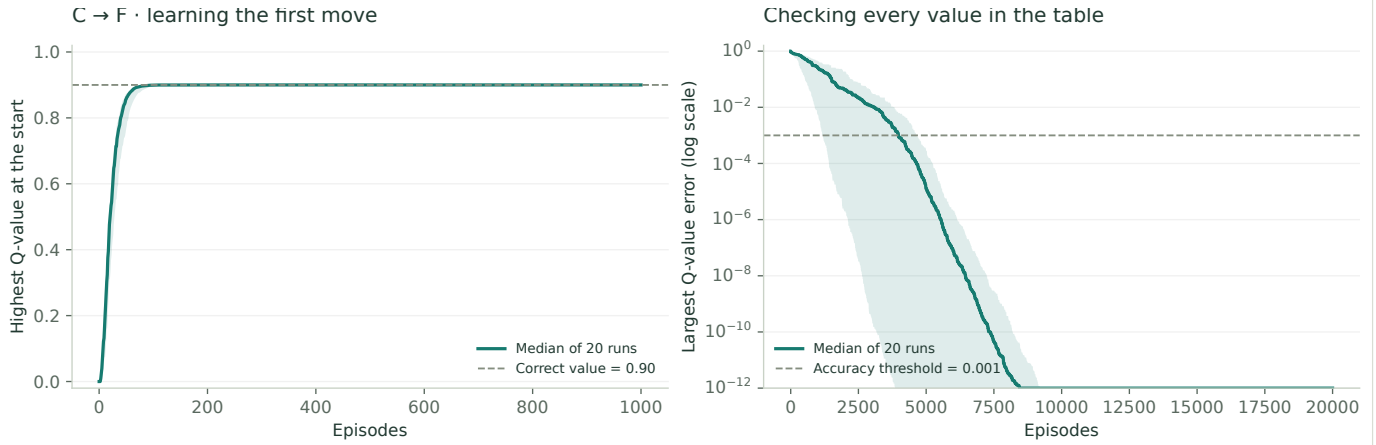


Figure 1. C → F. First panel: best Q-value at the start during the first 1,000 episodes. Second panel: largest absolute error in the whole table over the full 20,000 episodes, on a logarithmic scale. Lines show medians and bands show the 10th–90th seed percentiles. Numerical plot data · Full-size figure.

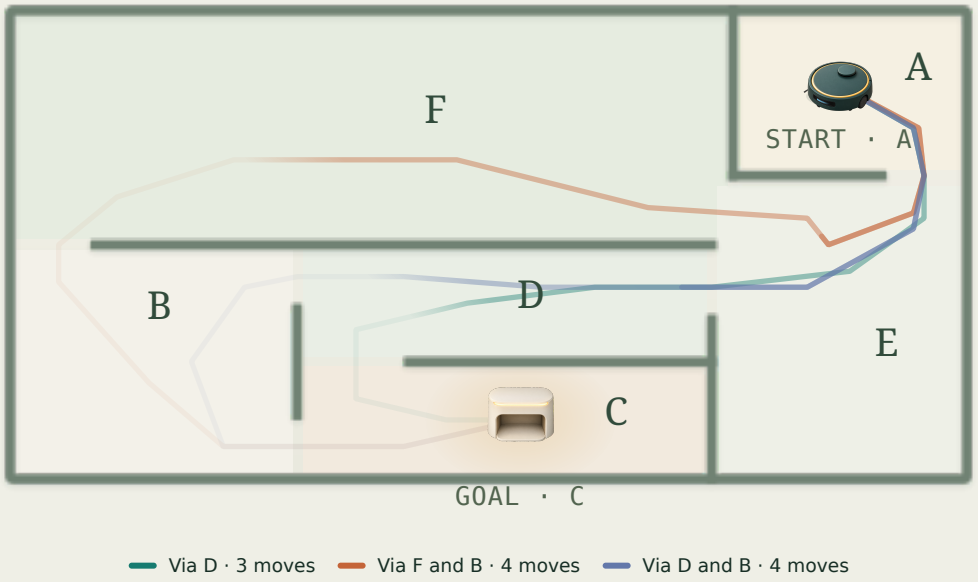
What the evidence says. All 20 runs learned a shortest 2-move route and passed the whole-table accuracy check. The median accuracy-check episode was **3,974**; the middle 80% of seeds passed between episodes 1,180.5 and 4,639.5. The curves therefore show both learning towards the correct values and variation in how quickly different runs get there.

▾ Exact final error and action coverage

The largest final error across these 20 seeds is 1.33e-15, approximately numerical rounding precision. Every allowed state-action pair was visited; the least-visited pair across all seeds received 765 updates. Tables above are rounded for reading; the CSVs retain full precision.

EXPERIMENT 2

What changes when the robot starts in A and aims for C?



Experiment 2 · the same map, from A to C. The robot stays in A and the destination in C. Teal shows A → E → D → C (3 moves); orange goes via F and B (4 moves); blue goes via D and B (4 moves). Moving shadows fade as they reach C. These are illustrative legal alternatives, not recorded training trajectories. The learned seed-0 route is reported below.

I reset all values to zero and train again. This time, entering C earns the reward and ends the episode. Notice that C is now the empty terminal row; F becomes an ordinary room with available moves. These are the saved initial and final tables from seed 0.

Before learning

Initial Q-table · episode 0 · A → C · seed 0

S / A	A	B	C	D	E	F
A	–	–	–	–	0	–
B	–	–	0	0	–	0
C	–	–	–	–	–	–
D	–	0	0	–	0	–
E	0	–	–	0	–	0
F	–	0	–	–	0	–

After learning

Final Q-table · episode 20,000 · A → C · seed 0

S / A	A	B	C	D	E	F
A	–	–	–	–	0.810	–
B	–	–	1.000	0.900	–	0.810
C	–	–	–	–	–	–
D	–	0.900	1.000	–	0.810	–
E	0.729	–	–	0.900	–	0.810
F	–	0.900	–	–	0.810	–

> Now read row **E**: moving to D has value **0.900**, compared with **0.810** for F and **0.729** for A. The robot should head towards D, then enter C.

Values are rounded to three decimals; zeros are shown as 0. Full-precision files: initial table · final table · visit counts.

Following the highest learned values gives this route:

```
A → E → D → C
```

It takes **3 moves**, matching a shortest route in the map. This is a greedy evaluation: the robot follows its learned values without deliberately exploring. Exact ties during evaluation use the alphabetical rule; tiny numerical differences can choose between equally short routes.

Did the values settle down?

The first panel shows the best estimate for the first move rising from zero towards **0.81**. That is the correct discounted value of reaching the goal in 3 moves. The second panel checks *every allowed move*, including moves away from this route: as its error falls, the whole table becomes more accurate. Its dashed line marks my 0.001 accuracy threshold.

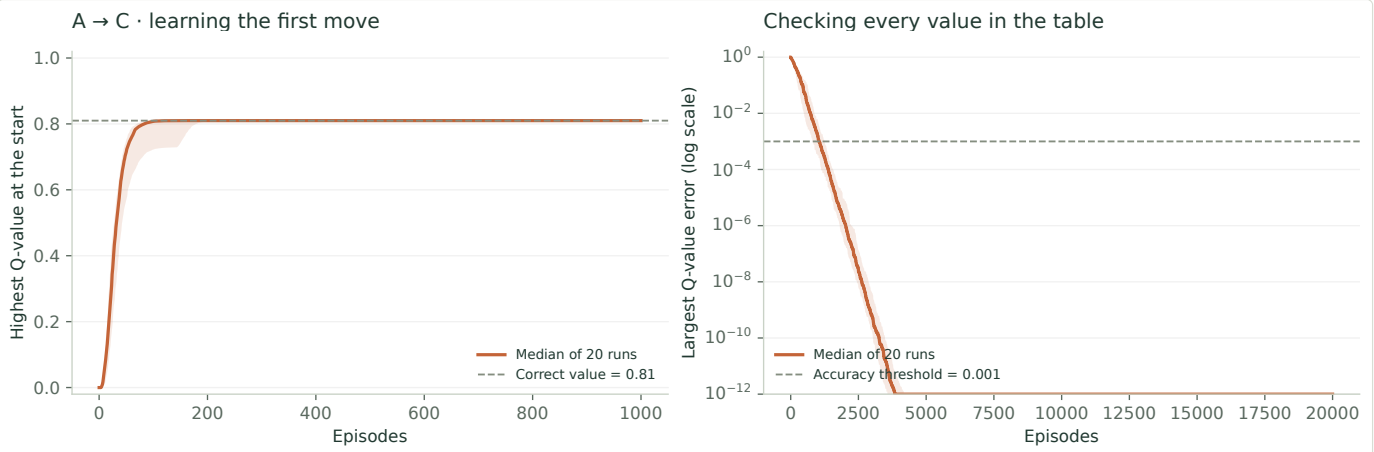


Figure 2. A → C. First panel: best Q-value at the start during the first 1,000 episodes. Second panel: largest absolute error in the whole table over the full 20,000 episodes, on a logarithmic scale. Lines show medians and bands show the 10th–90th seed percentiles. Numerical plot data · Full-size figure.

What the evidence says. All 20 runs learned a shortest 3-move route and passed the whole-table accuracy check. The median accuracy-check episode was **1,043**; the middle 80% of seeds passed between episodes 806.6 and 1,231.9. The curves therefore show both learning towards the correct values and variation in how quickly different runs get there.

▾ Exact final error and action coverage

The largest final error across these 20 seeds is 1.78e-15, approximately numerical rounding precision. Every allowed state-action pair was visited; the least-visited pair across all seeds received 1,312 updates. Tables above are rounded for reading; the CSVs retain full precision.

▾ Reference values and convergence criterion

The exact reference is $Q^*(s,a) = 0.9^{d(a,g)}$, where d is the shortest distance from the room reached by action a to goal g. An action entering the goal has d = 0, so its exact value is 1. Starting two moves from F gives a first-action value of 0.9; starting three moves from C gives $0.9^2 = 0.81$. The immediate reward is discounted by the zeroth power, so a route with n moves has first-action value 0.9^{n-1}.

I calculate distances with breadth-first search and check the reference independently with value iteration. The full-table error after episode e is $E_e = \max_{(s,a)\ \text{valid}} |Q_e(s,a) - Q^*(s,a)|$. The error panel uses a logarithmic vertical axis: equal vertical steps represent factors of ten. Values below 10^{-12} are drawn at that floor; the CSVs retain their actual precision.

The accuracy episode is the beginning of the final below-threshold sequence, provided that sequence lasts at least 100 episodes. This is checked after the fixed run has finished. It is not an early-stopping rule.

▾ Inspect how the saved Q-tables changed during learning

PART 04

Comparison and critical analysis

Moving the goal changes what a good move is.

With F as the goal, a robot in D should move towards B or E. With C as the goal, it should move directly to C instead. The reward and stopping point have changed, so the robot needs a different Q-table. F now has outgoing actions, while C becomes the terminal row.

With the B–C doorway, C → B → F takes two moves, while A → E → D → C takes three. The exact best first-action values are therefore **0.90 for C → F** and **0.81 for A → C**. The original route reaches its reward sooner, so it has less discounting.

The whole-table accuracy check was reached at a median episode of **3,974 for C → F** and **1,043 for A → C**. Under these settings, A → C learned the full table sooner on that measure. The shaded bands show randomness across seeds; these medians are not guaranteed learning times.

Changing only a start would change the experiences collected during training, but not the exact optimal values for a fixed goal. Here I changed both the start and goal together, as required. Their combined effect cannot tell me which change caused the learning-speed difference; the further experiments below investigate that distinction.

PART 04 · GOING FURTHER

Additional investigations

The two required experiments above complete the assignment’s core comparison. The following two questions are my additional investigations. Their answers are visible here; the supporting tables, figures and discussion follow below.

ADDITIONAL QUESTION 1

Do equally short routes lead to equally fast learning?

No, not in these experiments. C → F and A → F both take 2 moves, yet their median whole-table accuracy episodes are **3,974** and **5,522.5**. From B, both targets take 1 move, yet the medians are 7,910.5 for F and 4,280 for C.

What this means: route length alone does not predict how quickly all action values are learned. Starting position and terminal goal alter which actions get sampled. This is a descriptive result for this graph and protocol, not a universal speed ranking.

Read the evidence ↓

ADDITIONAL QUESTION 2

Can a good route coexist with inaccurate values?

Yes. Without forced exploration (ε = 0), **14/20** runs learn a shortest route, and the other 6 still reach F by longer routes. Yet **0/20** pass the full-table accuracy check: final errors remain 0.9–1.0.

What this means: a successful route and a nearly unchanging table do not establish accurate values for untried alternatives. Keep exploring and check action coverage and whole-table error. In the ε = 0.5 comparison, all 20 runs learn shortest routes and pass the accuracy check.

Read the evidence ↓

▼ Route length and whole-table learning speed

To separate start and target effects, I add A → F, B → F, and B → C. Together with A → C, these compare starts A/B with targets F/C while holding the other setting fixed. I use B rather than C as the second start because C → C would finish before the robot takes a move.

Read the table as a comparison of learning speed. “Accuracy episode” tells me when the entire table passes the 0.001 check. The range shows variation between seeds. “Transitions” counts actual moves, since equal episode budgets need not involve equal numbers of moves. The visit count shows how much attention the least-sampled action received.

START → TARGET	ACCURACY EPISODE: MEDIAN [P10, P90]	MEDIAN TRANSITIONS	FEWEST VISITS, ANY SEED	SHORTEST ROUTE
C → F	3,974 [1,180.5, 4,639.5]	68,472.5	765	2 steps
A → F	5,522.5 [4,708, 5,846.6]	61,245.5	334	2 steps

START → TARGET	ACCURACY EPISODE: MEDIAN [P10, P90]	MEDIAN TRANSITIONS	FEWEST VISITS, ANY SEED	SHORTEST ROUTE
B → F	7,910.5 [6,753.7, 8,532.7]	42,500.5	353	1 steps
A → C	1,043 [806.6, 1,231.9]	91,806.5	1312	3 steps
B → C	4,280 [3,483.5, 4,540.1]	37,500.5	370	1 steps

In the figure, each dot is one run. Farther right means more episodes were needed to learn the whole table accurately. The short orange mark is the median; the pale horizontal span covers the middle 80% of seeds.

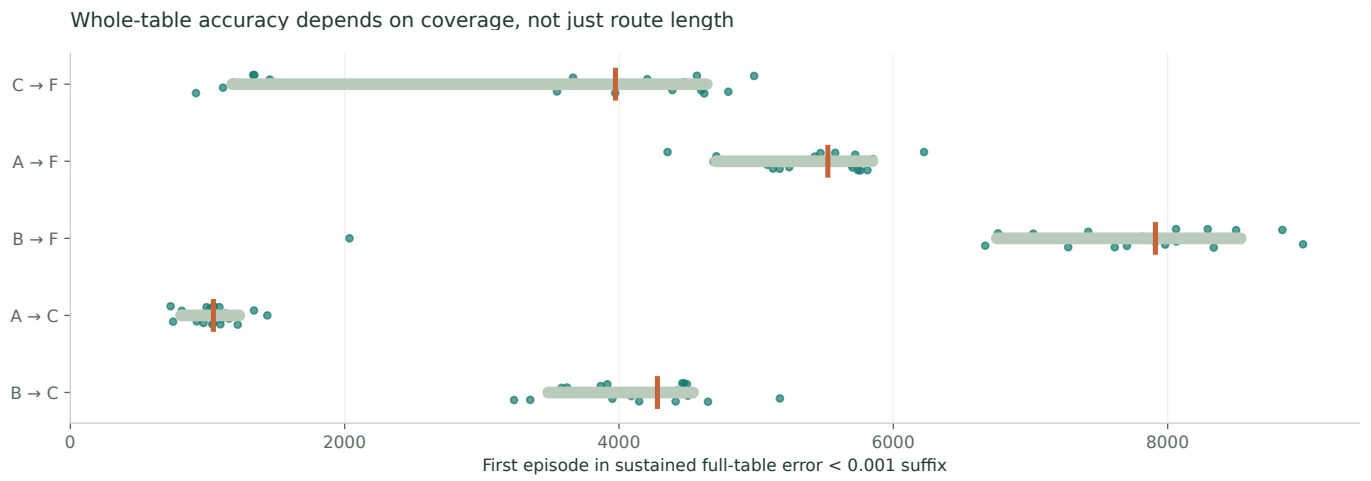


Figure 3. Episodes needed for sustained full-table error below 0.001. All five configurations use the same learning settings and 20 seeds. This measures coverage of the entire table, rather than only the length of the final route. Full-size figure.

Equal route lengths can produce different learning times.

Holding the goal at F, both starts C and A are two moves away. Yet median whole-table accuracy occurs at 3,974 and 5,522.5 episodes respectively. The descriptive seed-matched difference, A minus C, is 1,474 [502, 4,680.4] episodes (median [P10, P90]). Which branch an episode starts in affects which actions are sampled. The reference Q-table is identical for these two starts; the trajectories used to learn it differ.

Equal distances do not make the goals equivalent.

From B, both F and C can now be reached in one move. The median whole-table accuracy episodes are nevertheless 7,910.5 for B → F and 4,280 for B → C. Changing the goal changes which transitions receive a reward and terminate the episode, affecting subsequent coverage. These comparisons show why shortest-route length alone does not determine whole-table learning speed; they do not establish a universal rate.

The results show why shortest-path length alone is not enough to predict whole-table learning speed: a short successful episode may rarely reach distant branches. These are descriptive comparisons in one small graph, with no significance claim or claim of a universal relationship. Seed-level data are in summary.csv.

▼ Exploration and value accuracy

I repeat C → F with **ε = 0**, keeping every other main setting fixed. The robot no longer deliberately tries random alternatives, although exact ties are still broken randomly. Once a positive-valued route emerges, other moves may receive little attention.

Here I distinguish **accuracy** from **change**. Accuracy asks whether the values are right. Episode change asks how much the table changed during one attempt. An incorrect value can stay almost unchanged if the robot rarely revisits it.

In these saved runs, **14/20 final greedy policies take a shortest route**. Nevertheless, final full-table errors range from **0.900 to 1.000**, and 1–10 valid action pairs per run were never visited. No run meets the full-table tolerance. Episode changes remain below 0.001 from a median episode of **63** onwards for at least 100 episodes through the run end.

Compare the two panels below. In the first, a high orange line means large errors remain without forced exploration. In the second, a low orange line means the table is barely changing. Seeing both at once explains how a flat learning curve can give a misleading impression of convergence.

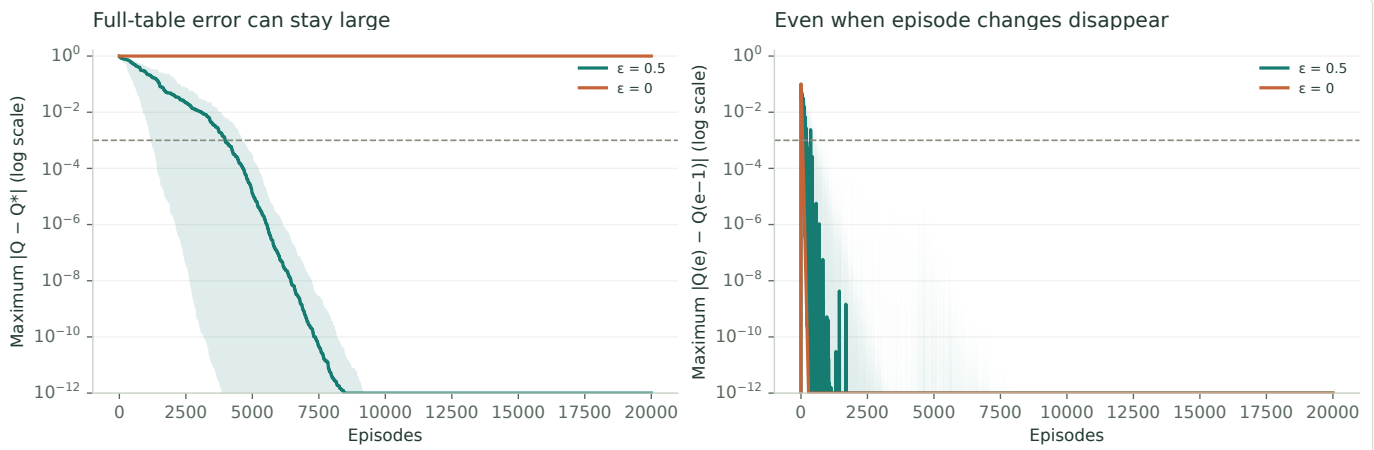


Figure 4. C → F with ε = 0.5 versus ε = 0. Left: largest full-table error. Right: largest absolute Q-table change in one episode. Lines are medians and bands span the 10th–90th seed percentiles. Both vertical axes are logarithmic, with a display floor of 10^{-12}. Full-size figure.

The lesson is that a successful route, a stable table, and an accurate table are different outcomes. I therefore also retain action visit counts and the Bellman residual, which checks whether the table agrees with a one-step optimal backup of the known model. In the two required cases, the largest final residual across all 40 runs is 5.55e-16, and every valid pair has at least 765 visits.

▼ Pilot settings and training budget

A retained pilot used ε = 0.2 and 3,000 episodes. All 20 runs in each required case found a shortest route, but the worst full-table errors were 0.900 (C → F) and 0.349 (A → C). Some actions had received too few updates. I increased ε to 0.5 and the budget to 20,000 episodes to improve coverage.

Both settings changed, so this pilot is a reason for the final protocol, not a controlled estimate of either setting's individual effect. All pilot outputs and their source provenance remain in `experiments/pilot_3000/`.

PART 05

Takeaways and references

Q-learning learns useful action values by revising estimates after sampled moves. In both required experiments, the learned greedy policy reaches the target by a shortest route, and the complete table passes my declared accuracy check across all 20 seeds.

Changing from C → F to A → C changes which actions deserve high values and which room ends an episode. The corrected map gives a two-move original route and a three-move modified route, with different policies and first-action values. The plots show how the estimates develop; repeated runs show how much the learning process varies with randomness.

These conclusions apply to this small deterministic room graph and the stated settings. The finite runs with constant α = 0.1 do not prove the classical asymptotic convergence result, which also requires repeated sampling and suitable step-size conditions [2]. Larger or noisy environments, other parameters, and other room-graph interpretations could behave differently.

References and attribution

1. Sutton, R. S. & Barto, A. G. *Reinforcement Learning: An Introduction*, 2nd ed., Chapter 6.5, Q-learning. Author's book page ↗
2. Watkins, C. J. C. H. & Dayan, P. (1992). Q-learning. *Machine Learning* 8, 279–292. Author's paper page ↗
3. Yu, J. (Fall 2026). *Mid-Term Assignment: Implementing Q-Learning*, IOTA 5201. Course-provided brief. My room graph and original C → F interpretation are based on Figure 1. The exact experimental diagram is drawn from my declared graph; both conceptual floorplans reuse the corrected experimental doorways. Their traces illustrate legal alternatives rather than recorded trajectories; robot and dock cutouts are AI-generated.
4. GitHub Docs. Configuring a publishing source for GitHub Pages ↗. Hosting workflow only.

AI use statement

OpenAI Codex was used to implement the Python code, run experiments, create figures, build the HTML report and reproduction tools, and draft explanations. The built-in image-generation tool produced the robot and dock artwork (prompts).

I used AI as an implementation and drafting tool while defining the report structure and presentation, revising the experimental map, and reviewing and refining the explanations and visuals. Outputs were checked using numerical tests, independent reference values, saved-result verification, seed replay and browser checks.

SUPPORTING FILES

Report & files

This webpage is the technical report. The same `report/index.html` is published online and included in the submission ZIP. The PDF is a printable companion.

To read the project, open `report/index.html`. To revise its text, edit `code/report_template.html`, then run `python scripts/build_report.py`. The generator inserts saved experimental values and recreates the figures.

Source repository · clzJY/iota-qlearning ↗
Download the submission ZIP ↗ · Read the PDF
Experiment settings · All-run summary

Reproduce the experiments

The ZIP includes source, dependency versions, seeds, configurations, numerical plot data, initial/final tables, and complete traces. Run these commands from the project root:

```
python -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements-lock.txt
python -m unittest discover -s code -p 'test_*.py'
python code/run_experiments.py
python scripts/build_report.py
python scripts/verify_project.py --replay
python scripts/package_submission.py
```

The README explains running either required configuration separately, optional PDF export, and opening the HTML directly or with a simple local server. Reading the report requires no account, external service, or training run. Online references and release downloads require an internet connection.

ROBOT NAVIGATION

Chenglong Zhang · 50068179
IOTA 5201 · Fall 2026